

Verifier-Guided Human–AI Search for Six van der Waerden Lower Bounds

John O’Laughlin
Independent researcher

July 2026

Abstract

We give explicit colorings establishing

$$W(7, 3) > 344$$

and

$$\begin{aligned} w(2; 3, 32) &> 1011, & w(2; 3, 33) &> 1069, & w(2; 3, 35) &> 1205, \\ w(2; 3, 36) &> 1259, & w(2; 3, 39) &> 1420. \end{aligned}$$

Here and throughout, notation is *color first*: $W(r, k)$ concerns r colors and monochromatic k -term arithmetic progressions, while $w(r; k_1, \dots, k_r)$ assigns forbidden length k_i to color i . The first result is therefore a 7-coloring of $[1, 344]$ with no monochromatic 3-term progression, not a 3-coloring avoiding 7-term progressions. The mixed results are 2-colorings in which color 1 avoids 3-term progressions and color 2 avoids t -term progressions.

The search combined focused stochastic local search with clause-weight redistribution and phase-seeded conflict-driven clause learning. Most mixed improvements were obtained by extending existing certificates, while one required a two-variable coordinated repair found by a SAT solver. Every claim is represented by a complete coloring and checked by a small, standalone verifier that shares no code with the search programs. We also describe the role of large-language-model assistants in literature triage, software development, and experiment orchestration. Generated prose or solver status was never treated as evidence: the governing rule was *certificate or nothing*.

1 Introduction

For positive integers r and k , van der Waerden’s theorem says that a sufficiently long interval, under every r -coloring, contains a monochromatic arithmetic progression of length k . Determining the least such interval length is difficult even for small parameters. Computational lower bounds are naturally constructive: one exhibits a coloring of a long interval that avoids all forbidden progressions.

Explicit constructions, local search, and propositional satisfiability have all contributed to this program [5, 1, 6, 10, 8]. Ahmed, Kullmann, and Snevily studied the mixed family $w(2; 3, t)$, proving $w(2; 3, 19) = 349$ and publishing lower-bound certificates through $t = 39$ [1]. Komkov later published SAT-derived certificates for several classical parameters, including $W(7, 3) > 343$ [8]. The constructions in this paper improve five of the mixed bounds stated by Ahmed–Kullmann–Snevily and the $W(7, 3)$ bound stated by Komkov.

The contributions are:

1. six complete, verifier-accepted colorings giving the bounds in the abstract;
2. a hybrid search pipeline in which stochastic local search supplies strong phase information to a complete SAT solver;
3. a small code-independent verifier, independent test generation, and cryptographic hashes for the certificates; and
4. a transparent account of an interactive human–AI computational mathematics workflow and its epistemic safeguards.

The inequalities themselves do not depend on a novelty search: each follows from its displayed finite certificate. Priority is a separate bibliographic claim. To the best of our literature search, the bounds improve the cited published baselines; the companion repository preserves the source transcriptions used in that comparison.

The priority audit was last updated on July 23, 2026. It checked the Ahmed–Kullmann–Snevily and Komkov source versions used for the baselines, Monroe’s version-7 tables, the Rabung–Lotts construction paper, the *Leaps in Bounds* tracker, and exact-formula and title searches across arXiv and the web through that date [1, 8, 10, 12, 9]. This records the scope of the priority claim without making certificate validity depend on bibliographic completeness.

2 Notation and the physical meaning of the claims

An arithmetic progression of length k , abbreviated k -AP, is

$$a, a + d, \dots, a + (k - 1)d$$

with $d \geq 1$. Write $[1, n] = \{1, \dots, n\}$.

- $W(r, k)$ is the least n such that every coloring $c : [1, n] \rightarrow \{1, \dots, r\}$ contains a monochromatic k -AP.
- $w(r; k_1, \dots, k_r)$ is the least n such that every such coloring has, for some color i , a k_i -AP entirely in color i .

Thus $W(r, k) = w(r; k, \dots, k)$. This paper always puts the number of colors first. Some literature reverses the two arguments of the classical quantity. Under that alternative convention, our $W(7, 3)$ can be written $W(3, 7)$. To make an argument-order mistake impossible to hide, we pair every formula with its physical object:

$W(7, 3) > 344$ means a 7-coloring of $[1, 344]$ containing no monochromatic 3-term arithmetic progression.

A concrete collision is worth spelling out. In length-first notation, $W(7, 3)$ denotes the quantity written here as $W(3, 7)$: three colors avoiding 7-term progressions. Its known lower bound $W(3, 7) > 48,811$ is unrelated to the 7-color, 3-term result of this paper [12, 9].

Likewise, $w(2; 3, 32) > 1011$ means a 2-coloring of $[1, 1011]$ in which color 1 has no 3-AP and color 2 has no 32-AP. The standalone verifier prints both the parameter tuple and the color-first formula, and regression tests lock this wording.

3 Certified results

Theorem 1. *There is a 7-coloring of $[1, 344]$ with no monochromatic 3-AP. There are also 2-colorings of $[1, n_t]$ in which color 1 contains no 3-AP and color 2 contains no t -AP, for*

$$(t, n_t) \in \{(32, 1011), (33, 1069), (35, 1205), (36, 1259), (39, 1420)\}.$$

Consequently, in color-first notation,

$$\begin{aligned} W(7, 3) &> 344, \\ w(2; 3, 32) &> 1011, & w(2; 3, 33) &> 1069, \\ w(2; 3, 35) &> 1205, & w(2; 3, 36) &> 1259, \\ w(2; 3, 39) &> 1420. \end{aligned}$$

Table 1: Certified improvements over the cited baselines. “Gain” is the increase in the length of the exhibited coloring, not a claim about the unknown exact value.

Color-first quantity	Cited bound	Bound here	Gain
$W(7, 3)$	> 343	> 344	1
$w(2; 3, 32)$	> 1006	> 1011	5
$w(2; 3, 33)$	> 1063	> 1069	6
$w(2; 3, 35)$	> 1204	> 1205	1
$w(2; 3, 36)$	> 1257	> 1259	2
$w(2; 3, 39)$	> 1418	> 1420	2

The cited mixed baselines are the certificate lengths in the appendix of Ahmed–Kullmann–Snevily [1]; the classical baseline is Komkov’s certificate [8]. For $t = 31, 34, 37, 38$, our searches did not improve the corresponding mixed baselines 930, 1143, 1338, 1378. This negative statement describes the run outcome only and makes no mathematical claim about those four unknown values.

4 Certificates and verification

4.1 Certificate format

A certificate is a whitespace-separated sequence

$$r; \quad k_1, \dots, k_r; \quad n; \quad c_1, \dots, c_n,$$

where $c_j \in \{1, \dots, r\}$ is the color of j . Comment lines are allowed, but the verifier rejects missing colors, extra integer tokens, out-of-range colors, and malformed parameters. Exit status 0 means valid, 1 means a forbidden progression was found, and 2 means malformed input.

4.2 Exhaustive checker

For each possible first element a , the verifier takes $i = c_a$, sets $k = k_i$, and enumerates

$$1 \leq d \leq \left\lfloor \frac{n-a}{k-1} \right\rfloor.$$

It rejects if all of

$$c_a, c_{a+d}, \dots, c_{a+(k-1)d}$$

equal i .

Lemma 1. *If the verifier accepts a certificate with parameters $(r; k_1, \dots, k_r; n)$, then the certificate establishes $w(r; k_1, \dots, k_r) > n$.*

Proof. Suppose a forbidden monochromatic progression exists. Let a be its first term, d its common difference, and $i = c_a$ its color. Because the progression lies in $[1, n]$, the verifier’s loop at a includes this d . It then checks exactly k_i terms and finds all of them colored i , so it rejects. The contrapositive gives the claim. $\square$

Proof of Theorem 1. The six complete colorings named in Table 2 are included in the companion repository. Each was accepted by the standalone verifier at the following commit:

bd16953d517f57935574b15d60f7c5715a9d5c75

Each file matches the SHA-256 digest in that table. Apply Lemma 1. $\square$

Implementation independence is intentional: `verifier.c` shares no code with a generator, searcher, model decoder, or source-expansion tool. An independent Python program generates eight small regression instances covering valid, invalid, mixed-length, swapped-length, malformed, and $k = 2$ cases. The repository’s full check also regenerates 12 published construction certificates byte for byte, accepts all 34 tracked complete certificates, and validates the certificate hash manifest. These tests do not replace the soundness argument above, but they reduce the risk of a parser or implementation error.

5 Search encodings and pipeline

5.1 Implicit Boolean encoding for $w(2; 3, t)$

Let $x_j = 1$ mean that j has color 1, the color that must avoid 3-APs. Every 3-AP, (a, b, c) , induces the negative clause

$$\neg x_a \vee \neg x_b \vee \neg x_c.$$

Every t -AP, $(a_1, \dots, a_t)$, induces the positive clause

$$x_{a_1} \vee \dots \vee x_{a_t}.$$

The searcher stores progressions and their incidence with elements, but does not materialize a general-purpose SAT object on its hot path. It maintains violation counts incrementally after a flip.

5.2 Uniform encoding for $W(7, 3)$

For the 7-color instance, variables $y_{j,q}$ encode that element j has color q . Exactly-one constraints are imposed for every element, and every 3-AP (a, b, c) and color q contributes

$$\neg y_{a,q} \vee \neg y_{b,q} \vee \neg y_{c,q}.$$

Search states use a direct one-color-per-element representation; the one-hot encoding is emitted only for the CDCL lane.

5.3 Focused local search

The stochastic lane combines mixed random-walk ideas [14] with DDFW-style redistribution of clause weights [7]. A step focuses on a currently violated progression, scores recolorings using the affected incidence lists, and makes either a greedy or noisy move. Restarts include published-certificate seeds, reversals, cyclic and residue templates, palindromic instances, and random starts. Independent workers explore different restart classes and seeds. Best-so-far states are explicitly marked as *not certificates*.

5.4 Phase-seeded CDCL

Local search often reached assignments with one violated progression but could not coordinate the several flips needed to escape. We therefore emitted a conventional CNF while polarity-transforming each primary variable so that the solver’s default phase represents the supplied search state. Kissat 4.0.4 [3] was used for the decisive full-instance runs; CaDiCaL 3.0.1 [2] provided an API lane for Hamming-ball, fixed-window, reflection, and small-instance cross-checks. Kissat and CaDiCaL are candidate generators only. A decoded model is not a result until the standalone verifier accepts it.

Reflection lex leaders were used only where their safety followed from the unseeded symmetry. Palindromic and fixed-window constraints were treated as streamlining: satisfiable output still gives an unconditional lower bound, but an unsatisfiable streamlined instance says nothing about the unrestricted problem. No unsatisfiability claim is made in this paper.

6 How the six certificates were found

6.1 The $W(7, 3)$ “wall”

Starting from Komkov’s coloring at $n = 343$, repeated DDFW runs at $n = 344$ reached assignments with a single violation. Across independent basins, that violation was always

$$\{2, 173, 344\},$$

the 3-AP with common difference 171. This repeated obstruction was useful phase information, not evidence of impossibility.

Unseeded Kissat did not settle even the known-satisfiable $n = 343$ encoding within ten minutes in our calibration. With seed-relative polarities, the same calibration completed in under one second. A phase-seeded run from one of the one-violation $n = 344$ basins then returned a model. The accepted coloring differs from that basin at 25 positions, distributed across the interval, and recolors element 344 itself. The experiment therefore supports a specific hybrid account: local search located a strong basin, and CDCL assembled a globally coordinated escape that single-flip search had not found.

At $n = 345$, two further local-search chains reached the same remaining violation $\{3, 174, 345\}$, also with difference 171, but bounded phase-seeded CDCL runs returned no decision. This is reported only as a reproducible search observation. It is neither evidence of unsatisfiability nor a determination of $W(7, 3)$.

6.2 The $w(2; 3, t)$ family

The nine compact certificates for $t = 31, \dots, 39$ were transcribed from the Ahmed–Kullmann–Snevily appendix, mechanically expanded, and accepted by the standalone verifier before use as

seeds. For $t = 32, 33, 36, 39$, orientation and endpoint-color sweeps produced short extension ladders. Several steps required no interior change; one $t = 33$ step required a single local-search flip. The final accepted lengths were 1011, 1069, 1259, and 1420 respectively.

The $t = 35$ step illustrates the value of the CDCL handoff. A reversed seed at $n = 1205$ remained one violation away under the endpoint probe. Three phase-seeded Kissat configurations returned the same satisfying model in 0.05–0.06 seconds. Relative to the basin, the model flips exactly elements 892 and 1188. That two-variable coordination was unavailable to the one-flip endpoint repair, and all three decoded copies independently passed the verifier.

Bounded searches for the next lengths and for $t = 31, 34, 37, 38$ were undecided. In particular, a timeout is recorded as UNKNOWN, never as unsatisfiable.

7 Human–AI workflow and epistemic controls

Recent systems have coupled machine learning or large language models with mathematical evaluators, program search, or human interpretation [4, 13, 11]. The workflow here was less autonomous and more conversational. OpenAI Codex and Anthropic Claude were used interactively for literature triage, experiment design, C and Python implementation, process supervision, code review, and log synthesis. Google Gemini was used during a manuscript-stage check that exposed the danger of reversing color-first and length-first notation.

The human researcher chose the target family, set compute budgets, approved external actions, redirected unsuccessful work, and decided what evidence could support a public claim. The language-model systems proposed and implemented many of the concrete search steps. They are acknowledged as research tools rather than listed as authors because they cannot take responsibility for the manuscript or its claims.

Four controls separated useful assistance from mathematical evidence:

1. **Generator–verifier separation.** Search code and the verifier share no implementation.
2. **Complete witnesses.** A score, a nearly satisfying state, an LLM assertion, or a SAT timeout is not a result.
3. **Persistent provenance.** Contemporaneous logs record failed approaches, corrections, solver versions, seeds, and bounded UNKNOWN outcomes.
4. **Notation redundancy.** Formulas are paired with plain descriptions of the physical coloring, and verifier output repeats the color count and forbidden lengths before printing a formula.

This differs from FunSearch [13] and AlphaEvolve [11]: no learned or evolutionary controller maintained a population of programs. Instead, conversational agents helped build and operate a conventional, domain-specific SLS–SAT pipeline. The common principle is evaluator grounding. Here the evaluator is deliberately small, deterministic, and separate from every heuristic component.

8 Reproducibility

The companion repository is <https://github.com/olaugh/vdw-search>. At the verifier revision used for this paper, a verification-only reproduction requires a C11 compiler, POSIX `sh`, `make`, and Python 3:

```

make verifier
./verifier certs/RECORD_W7_3_gt344.txt
./verifier certs/m5_w2_3_32_gt1011.txt
make check

```

The final command rebuilds dependency-free programs, regenerates independent test fixtures, runs the verifier and incremental-state tests, reproduces published-source certificates, verifies all tracked certificates, and checks their SHA-256 digests. The intermediate accepted certificates in the successful $t = 32, 33, 36, 39$ extension ladders are also tracked under `certs/` and covered by the same command.

Search binaries were compiled with C99/C11 compilers and hot runs used `-O3 -march=native`; a profile-guided build was also exercised for the mixed local-search lane. These optimizations affect search speed, not certificate meaning. CaDiCaL and Kissat are not required for verification. The exact solver releases used for the final search lane were CaDiCaL 3.0.1 and Kissat 4.0.4 (release commit `8af8e56`); the latter passed its bundled test suite before use.

9 Limitations and open questions

The paper establishes lower bounds only. It supplies no proof of unsatisfiability at the next interval length and therefore no exact value. The repeated common-difference-171 obstruction near $W(7, 3)$ is an empirical feature of sampled basins, not a theorem about all colorings. Likewise, unsuccessful mixed-family searches do not rank the true difficulty of the remaining parameters.

The verifier is small and tested but not formally verified. Its source, independent fixtures, complete witnesses, and hashes are supplied so that others can inspect it, write a second checker, or verify the progressions directly. An archival release should freeze the paper, source, certificates, and manifest together.

Finally, the literature search and manuscript benefited from fallible language models. This is a reason for stronger bibliographic and notation checks, not a reason to weaken them. The certificate claims are independent of those tools; the priority claim remains subject to ordinary scholarly correction if an earlier certificate is located.

Acknowledgments and AI-use statement

OpenAI Codex and Anthropic Claude materially assisted with code, literature triage, search orchestration, and manuscript preparation. Google Gemini contributed to a notation sanity check. All model outputs were reviewed in context, and no model output was accepted as mathematical evidence without a complete coloring passing the standalone verifier. The author accepts responsibility for the paper.

A Headline certificate files and hashes

Table 2: Complete files supporting [Theorem 1](#). Hashes are SHA-256.

Certificate file	SHA-256
certs/RECORD_W7_3_gt344.txt	d9f008e1a8985d51c70b8875b12df7520ad83ac4659670ef770493f95d1a fb22
certs/m5_w2_3_32_gt1011.txt	abe37f598351ac49f07e31a819c3ca87cdfd86d1537b9988613a18dc3c7d f8cc
certs/m5_w2_3_33_gt1069.txt	ddcb28d7aec8a88ca572f1da9cfcd3ffa78fff59463a7e7b09332c9069e2 9bcf
certs/m5_w2_3_35_gt1205.txt	de399156d8673b1c241cf22db41b3eb968e150e2f68478760b0eeaba068d 9d61
certs/m5_w2_3_36_gt1259.txt	182465150650d344bac38accce1187789a5c0561100ff9cd55a3534ff472 d2de
certs/m5_w2_3_39_gt1420.txt	11e5cdef316bdb2ad107921786c599553fceb60b33d68d37236700de63c8 7a88

References

- [1] Tanbir Ahmed, Oliver Kullmann, and Hunter Snevily. On the van der Waerden numbers $w(2; 3, t)$. *Discrete Applied Mathematics*, 174:27–51, 2014. <https://doi.org/10.1016/j.dam.2014.05.007>; arXiv:1102.5433.
- [2] Armin Biere, Tobias Faller, Katalin Fazekas, Mathias Fleury, Nils Froleyks, and Florian Pollitt. CaDiCaL 2.0. In *Computer Aided Verification*, volume 14681 of *Lecture Notes in Computer Science*, pages 133–152. Springer, 2024. https://doi.org/10.1007/978-3-031-65627-9_7.
- [3] Armin Biere, Tobias Faller, Katalin Fazekas, Mathias Fleury, Nils Froleyks, and Florian Pollitt. CaDiCaL, Gimsatul, IsaSAT and Kissat entering the SAT Competition 2024, 2024. SAT Competition 2024: Solver, Benchmark and Proof Checker Descriptions, University of Helsinki Report B-2024-1, 8–10; Kissat release 4.0.4, <https://github.com/arminbiere/kissat/releases/tag/rel-4.0.4>.
- [4] Alex Davies, Petar Veličković, Lars Buesing, et al. Advancing mathematics by guiding human intuition with AI. *Nature*, 600:70–74, 2021. <https://doi.org/10.1038/s41586-021-04086-x>.
- [5] Peter R. Herwig, Marijn J. H. Heule, Paul van Lambalgen, and Hans van Maaren. A new method to construct lower bounds for van der Waerden numbers. *Electronic Journal of Combinatorics*, 14(1):R6, 2007. <https://doi.org/10.37236/925>.
- [6] Marijn J. H. Heule. Avoiding triples in arithmetic progression. *Journal of Combinatorics*, 8(3):391–422, 2017. <https://doi.org/10.4310/JOC.2017.v8.n3.a1>.
- [7] Abdelraouf Ishtaiwi, John Thornton, Abdul Sattar, and Duc Nghia Pham. Neighbourhood clause weight redistribution in local search for SAT. In *Principles and Practice of Constraint Programming – CP 2005*, volume 3709 of *Lecture Notes in Computer Science*, pages 772–776. Springer, 2005. https://doi.org/10.1007/11564751_62.

- [8] Alexey V. Komkov. New lower bounds for van der Waerden numbers, 2020. arXiv:1701.05603v5.
- [9] Leaps in Bounds. van der Waerden Number $W(3,7)$. <https://leapsinbounds.org/constants/van-der-waerden-3-7/>, 2026. Accessed July 23, 2026.
- [10] Daniel Monroe. New lower bounds for van der Waerden numbers using distributed computing, 2022. arXiv:1603.03301v7.
- [11] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery, 2025. arXiv:2506.13131.
- [12] John Rabung and Mark Lotts. Improving the use of cyclic zippers in finding lower bounds for van der Waerden numbers. *Electronic Journal of Combinatorics*, 19(2):P35, 2012. <https://doi.org/10.37236/2363>.
- [13] Bernardino Romera-Paredes, Mohammadamin Barekatin, Alexander Novikov, et al. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024. <https://doi.org/10.1038/s41586-023-06924-6>.
- [14] Bart Selman, Henry A. Kautz, and Bram Cohen. Noise strategies for improving local search. In *Proceedings of the Twelfth National Conference on Artificial Intelligence*, pages 337–343. AAAI Press, 1994. <https://cdn.aaai.org/AAAI/1994/AAAI94-051.pdf>.